Petra Chinsangaram

Mar 4, 2026

Foundations of Python Programming

Assignment 06

https://github.com/petra-cc/IntroToProg-Python-Mod06

Assignment 6: Functions and Classes

Introduction

In this assignment, I created a python script in which a user can enter student information, have student information displayed to them, and append student information to a pre-existing JSON file. This assignment was very similar to the previous week's assignment in outcome, but the script was organized very differently. For this assignment, I organized the code into functions and categorized the functions into classes. I then created my while loop, presenting the choices and calling my functions as they were needed.

Creating the Program

I began by opening the Assignment 6 starter script into PyCharm and updating the change log in the header. The script begins by importing the JSON module.

Defining Constants and Variables

The constants were the same as in the previous assignment: MENU and FILE_NAME. Because I would be using functions and would thus require far fewer global variables, I only defined two variables: menu_choice, an empty string, and students, and empty list.

Creating Classes

In order to organize my code according to the separation of concerns, I first created two classes: FileProcessor and IO. The former contains functions concerning interactions with my JSON file (i.e., reading from and writing to the file), while the latter contains functions concerning collection input from and providing output to the user. I added descriptive document strings to each class.

Having created my classes, I created an outline of my functions by defining each with def and setting its parameters according to the assignment, but temporarily ignoring each with pass. Before each function, I used the @staticmethod decorator. Within the FileProcessor class I defined read_data_from_file(file_name: str, student_data: list) and write_data_to_file(file_name: str, student_data: list). Within the IO class I defined output_error_messages(message: str, error: Exception = None), output_menu(menu: str), input_menu_choice(), output_student_courses(student_data: list), and input_student_data(student_data: list)

Creating Functions: Error Messages

The first function I created was to manage error messages (Figure 1). I started by creating this function because it would be utilized within my other functions. It was within the IO class, because its purpose is to output data to the user. The parameters were message: str and error: Exception = None. Setting a default value of None meant that the function could be called without an exception. This function prints the error message argument provided when the function is called, and if there is also an exception, prints a technical message for the exception.

```
@staticmethod
def output_error_messages(message: str, error: Exception = None):
    """
    This function displays a custom error message to the user

    :param message: string containing the custom error message
    :param error: exception called by try/except block
    :return: None
    """
    print(message)
    if error is not None:
        print("-- Technical Error Message -- ")
        print(error, error.__doc__, type(error), sep="\n")
```

Figure 1: Function to output error messages

Creating Functions: File Processing

Within the FileProcessing class I created two functions: one to read data from the file and the other to write data to the file (Figure 2). Both had parameters of the file name and student data. They functioned similarly to the Assignment 5 code reading and writing to file, but in the event of an exception error, I called the error messages function, adding a custom message as the argument and the temporary variable e as the error. Additionally, I ended the read file function by returning the student_data list.

```
@staticmethod
def read_data_from_file(file_name: str, student_data: list):
    """
    This function reads and stores the data from the JSON file

    :param file_name: string containing the name of the file
    :param student_data: list of dictionaries containing student data
    :return: list of dictionaries containing student data
```

```
    """
    try:
        file = open(file_name, "r")
        student_data = json.load(file)
    except Exception as e:
        IO.output_error_messages(message="Error: There was a problem "
                                 "with reading the file.\n"
                                 "Please check that the file exists "
                                 "and that it is in a json format.",
                                 error=e)
    finally:
        # Check if a file object exists and is still open
        if file is not None and file.closed == False:
            file.close()
    return student_data

@staticmethod
def write_data_to_file(file_name: str, student_data: list):
    """
    This function writes the current student data to the JSON file

    :param file_name: string containing the name of the file
    :param student_data: list of dictionaries containing student data
    :return: None
    """
    try:
        file = open(file_name, "w")
        json.dump(student_data, file, indent=2)
    except Exception as e:
        IO.output_error_messages(message="Error: There was a problem "
                                 "with writing to the file.\n"
                                 "Please check that the file "
                                 "is not open by another program.",
```

```
                                error=e)
    finally:
        # Check if a file object exists and is still open
        if file is not None and file.closed == False:
            file.close()
```

Figure 2: File processing functions

Creating Functions: Menu Input/Output

Within the IO class, I included functions to present the menu to the user and collect the user's choice (Figure 3). The former has a menu parameter, and simply prints the menu with some extra line breaks. The latter has no parameters, and returns the user's choice. Additionally, if the user inputs anything unexpected, a try/except block raises an exception and calls the error messages function. This makes the script simpler later on.

```
@staticmethod
def output_menu(menu: str):
    """
    This function displays the menu of choices to the user

    :param menu: string containing the menu
    :return: None
    """
    print()
    print(menu)
    print()

@staticmethod
def input_menu_choice():
    """
    This function collects the user's menu choice

    :return: string containing the user's menu choice
    """
    try:
        choice = input("What would you like to do: ")
        if choice not in ("1", "2", "3", "4"):
            raise Exception("Please, only choose 1, 2, 3, or 4")
    except Exception as e:
        IO.output_error_messages(message=e.__str__())
    return choice
```

Figure 3: Menu input and output functions

Creating Functions: Data Input/Output

The final two functions in the IO class deal with the student data itself–presenting the current data to the user and collecting new data from the user (Figure 4). The output function uses a student data parameter to format a string to present to the user. The input function collects the data, raising exceptions as needed, before formatting the data, appending it to the student data list, printing a message to the user, and returning the updated list.

```
@staticmethod
def output_student_courses(student_data: list):
    """
    This function displays the list of students to the user

    :param student_data: list of dictionaries containing student data
    :return: None
    """
    print("-" * 50)
    for student in student_data:
        print(f"Student {student["FirstName"]} {student["LastName"]} "
              f"is enrolled in {student["CourseName"]}")
    print("-" * 50)

@staticmethod
def input_student_data(student_data: list):
    """
    This function collects the student's
    first name, last name, and course name from the user

    :param student_data: list of dicts containing previous student data
    :return: list of dicts containing previous and new student data
    """

    try:
        student_first_name = input("Enter the student's first name: ")
        if not student_first_name.isalpha():
            raise ValueError("Error: The first name "
```

```
                            "should not contain numbers.")
        student_last_name = input("Enter the student's last name: ")
        if not student_last_name.isalpha():
            raise ValueError("Error: The last name "
                            "should not contain numbers.")
        course_name = input("Please enter the name of the course: ")
        student_dict = {"FirstName": student_first_name,
                        "LastName": student_last_name,
                        "CourseName": course_name}
        student_data.append(student_dict)
        print(f"You have registered {student_first_name} "
              f"{student_last_name} for {course_name}.")
    except ValueError as e:
        IO.output_error_messages(message=e.__str__(), error=e)
    except Exception as e:
        IO.output_error_messages(message="Error: There was a problem "
                                 "with your entered data.", error=e)
    return student_data
```

Figure 4: Data input and output functions

Calling the Functions

Having defined all of the necessary functions, I was able to actually carry them out (Figure 5). I began by calling my read data function with my file name constant and students variable as arguments and placing the returned data into my students variable.

I then opened my while loop, presenting the menu by calling the output menu function and collecting the response by placing the returned data from the input menu choice function into my menu choice variable. If the user chose 1, the returned data from the input student data function was placed into the students variable. If the user chose 2, I called the output student data function. If the user chose 3, I called the write data function as well as the output student data function. If the user chose 4, I broke out of the while loop.

```
# Extract the data from the file
students = FileProcessor.read_data_from_file(file_name=FILE_NAME,
                                             student_data=students)


# Present and Process the data
```

```
while (True):

    # Present the menu of choices and collect the user's choice
    IO.output_menu(menu=MENU)
    menu_choice = IO.input_menu_choice()

    # Input user data
    if menu_choice == "1":

        students = IO.input_student_data(student_data=students)
        continue

    # Present the current data
    elif menu_choice == "2":

        IO.output_student_courses(student_data=students)
        continue

    # Save the data to a file
    elif menu_choice == "3":

        FileProcessor.write_data_to_file(file_name=FILE_NAME,
                                         student_data=students)
        IO.output_student_courses(student_data=students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop
```

Figure 5: While loop and functions being called

Testing the Program

I tested my program in PyCharm (Figures 6-9) and the Command Prompt (Figures 10-13). Both functioned properly, resulting in an updated JSON file (Figure 14).

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


What would you like to do: 1
Enter the student's first name: Vic
Enter the student's last name: Vu
Please enter the name of the course: Python 100
You have registered Vic Vu for Python 100.
```

```
What would you like to do: 2
--------------------------------------------------
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Vic Vu is enrolled in Python 100
--------------------------------------------------
```

```
What would you like to do: 3
--------------------------------------------------
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Vic Vu is enrolled in Python 100
--------------------------------------------------
```

```
What would you like to do: 4
Program Ended
```

Figures 6-9: Program output in PyCharm

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


What would you like to do: 1
Enter the student's first name: Vic
Enter the student's last name: Vu
Please enter the name of the course: Python 100
You have registered Vic Vu for Python 100.
```

```
What would you like to do: 2
--------------------------------------------------
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Vic Vu is enrolled in Python 100
--------------------------------------------------
```

```
What would you like to do: 3
--------------------------------------------------
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Vic Vu is enrolled in Python 100
--------------------------------------------------
```

```
What would you like to do: 4
Program Ended
```

Figures 10-13: Program output in Command Prompt

```
[
  {
    "FirstName": "Bob",
    "LastName": "Smith",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Sue",
    "LastName": "Jones",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Vic",
    "LastName": "Vu",
    "CourseName": "Python 100"
  }
]
```

Figure 14: Resulting Enrollments.json file viewed in PyCharm

Summary

This assignment built upon the previous assignment by organizing the code into functions and classes. This creates a much simpler and more streamlined main body of the code, and allows for sections of code to be reused, reducing redundancy. I can see that this would be very beneficial as a script gets longer and more complex.